

Parte 2 – Desarrollar los siguientes puntos, ponderación de actividad 90%.

Indicaciones: Dar respuesta a cada una de las preguntas que se muestran a continuación, tomando como referencia los tópicos abordados en clase.

a) Definir con sus propias palabras, y de manera general, que es una función y cuando pueden ser usadas y en donde las condiciones no permiten el uso de una función.

Una función es como una caja mágica que hace algo específico. Usted le da algo (datos), hace su trabajo y le devuelve un resultado. Se usa cuando usted tiene que hacer lo mismo varias veces en un programa, así no tiene que repetir el código. Pero no use funciones si lo que hace es muy simple, porque sería como usar un martillo para clavar un alfiler: innecesario, prácticamente una función en pocas palabras es una transformación.

b) Liste y describa los elementos que componen a una función, de manera general.

Nombre: Es como el nombre de la caja mágica. Lo usas para llamarla.

Parámetros: Son los datos que le das a la caja para que haga su trabajo.

Cuerpo: Es el código que dice qué hace la caja.

Valor de retorno: Es lo que la caja te devuelve después de hacer su trabajo.

Tipo de retorno: Es el tipo de dato que la caja te devuelve (un número, un texto, etc.).

c) Para usted que es lo que se logra al implementar funciones anidadas.

Usar funciones anidadas es como tener una caja mágica dentro de otra caja. Sirve para organizar mejor el código y que no se mezcle todo. Por ejemplo, si usted tiene una función grande, puedes dividirla en funciones más pequeñas que solo se usen dentro de ella.

d) Para usted que significa el termino recursividad, use mínimo dos líneas de texto para describir su respuesta.

La recursividad es cuando una función se llama a sí misma, como un bucle pero más elegante.

Pero ojo, si no la usamos bien, puede causar un bucle infinito o gastar mucha memoria.

e) Exponga un ejemplo práctico en donde se usen funciones recursivas, código C++ o pseudocódigo, y explíquelos.

```
#include <iostream> // Incluye la librería estándar de entrada y salida (para usar cout y cin)
```

```
using namespace std; // Usa el espacio de nombres estándar (para no escribir std:: antes de cout o endl)
```

```
// Función recursiva para calcular el factorial
```

```
int factorial(int n) { // Define una función llamada "factorial" que recibe un entero "n"
```

```
    if (n == 0 || n == 1) { // Si "n" es 0 o 1, entra aquí (caso base)
```

```
        return 1; // Devuelve 1, porque el factorial de 0 y 1 es 1
```

```
    } else { // Si "n" no es 0 ni 1, entra aquí
```

```
        return n * factorial(n - 1); // Llama a la función "factorial" con "n-1" y multiplica el resultado por "n"
```